



Apache Spark™ examples

This page shows you how to use different Apache Spark APIs with simple examples.

Spark is a great engine for small and large datasets. It can be used with single-node/localhost environments, or distributed clusters. Spark's expansive API, excellent performance, and flexibility make it a good option for many analyses. This guide shows examples with the following Spark APIs:

- DataFrames
- SQL
- Structured Streaming
- RDDs

The examples use small datasets so they are easy to follow.

Spark DataFrame example

This section shows you how to create a Spark DataFrame and run simple operations. The examples are on a small DataFrame, so you can easily see the functionality.

Let's start by creating a Spark Session:

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("demo").getOrCreate()
```

Some Spark runtime environments come with pre-instantiated Spark Sessions. The `getOrCreate()` method will use an existing Spark Session or create a new Spark Session if one does not already exist.

Create a Spark DataFrame

Start by creating a DataFrame with `first_name` and `age` columns and four rows of data:

```
df = spark.createDataFrame(
    [
        ("sue", 32),
        ("li", 3),
        ("bob", 75),
        ("heo", 13),
    ],
    ["first_name", "age"],
)
```

Use the `show()` method to view the contents of the DataFrame:

```
df.show()

+-----+----+
|first_name|age|
+-----+----+
|      sue| 32|
|       li|  3|
|      bob| 75|
|      heo| 13|
+-----+----+
```

Now, let's perform some data processing operations on the DataFrame.

Add a column to a Spark DataFrame

Let's add a `life_stage` column to the DataFrame that returns "child" if the age is 12 or under, "teenager" if the age is between 13 and 19, and "adult" if the age is 20 or older.

```
from pyspark.sql.functions import col, when

df1 = df.withColumn(
    "life_stage",
    when(col("age") < 13, "child")
    .when(col("age").between(13, 19), "teenager")
    .otherwise("adult"),
)
```

It's easy to add columns to a Spark DataFrame. Let's view the contents of `df1`.

```
df1.show()
```

```
+-----+---+-----+
|first_name|age|life_stage|
+-----+---+-----+
|      sue| 32|      adult|
|       li|  3|      child|
|      bob| 75|      adult|
|      heo| 13|  teenager|
+-----+---+-----+
```

Notice how the original DataFrame is unchanged:

```
df.show()
```

```
+-----+---+
|first_name|age|
+-----+---+
|      sue| 32|
|       li|  3|
|      bob| 75|
|      heo| 13|
+-----+---+
```

Spark operations don't mutate the DataFrame. You must assign the result to a new variable to access the DataFrame changes for subsequent operations.

Filter a Spark DataFrame

Now, filter the DataFrame so it only includes teenagers and adults.

```
df1.where(col("life_stage").isin(["teenager", "adult"])).show()
```

```
+-----+---+-----+
|first_name|age|life_stage|
+-----+---+-----+
|      sue| 32|      adult|
|      bob| 75|      adult|
|      heo| 13|  teenager|
+-----+---+-----+
```

Group by aggregation on Spark DataFrame

Now, let's compute the average age for everyone in the dataset:

```
from pyspark.sql.functions import avg

df1.select(avg("age")).show()

+-----+
|avg(age)|
+-----+
|   30.75|
+-----+
```

You can also compute the average age for each `life_stage`:

```
df1.groupBy("life_stage").avg().show()

+-----+-----+
|life_stage|avg(age)|
+-----+-----+
|      adult|    53.5|
|      child|     3.0|
|   teenager|    13.0|
+-----+-----+
```

Spark lets you run queries on DataFrames with SQL if you don't want to use the programmatic APIs.

Query the DataFrame with SQL

Here's how you can compute the average age for everyone with SQL:

```
spark.sql("select avg(age) from {df1}", df1=df1).show()

+-----+
|avg(age)|
+-----+
|   30.75|
+-----+
```

And here's how to compute the average age by `life_stage` with SQL:

```
spark.sql("select life_stage, avg(age) from {df1} group by
life_stage", df1=df1).show()

+-----+-----+
|life_stage|avg(age)|
+-----+-----+
|      adult|    53.5|
|      child|     3.0|
|   teenager|    13.0|
+-----+-----+
```

Spark lets you use the programmatic API, the SQL API, or a combination of both. This flexibility makes Spark accessible to a variety of users and powerfully expressive.

Spark SQL Example

Let's persist the DataFrame in a named Parquet table that is easily accessible via the SQL API.

```
df1.write.saveAsTable("some_people")
```

Make sure that the table is accessible via the table name:

```
spark.sql("select * from some_people").show()
```

```
+-----+---+-----+
|first_name|age|life_stage|
+-----+---+-----+
|      heo| 13|  teenager|
|      sue| 32|    adult|
|      bob| 75|    adult|
|       li|  3|    child|
+-----+---+-----+
```

Now, let's use SQL to insert a few more rows of data into the table:

```
spark.sql("INSERT INTO some_people VALUES ('frank', 4, 'child')")
```

Inspect the table contents to confirm the row was inserted:

```
spark.sql("select * from some_people").show()
```

```
+-----+---+-----+
|first_name|age|life_stage|
+-----+---+-----+
|      heo| 13|  teenager|
|      sue| 32|    adult|
|      bob| 75|    adult|
|       li|  3|    child|
|    frank|  4|    child|
+-----+---+-----+
```

Run a query that returns the teenagers:

```
spark.sql("select * from some_people where  
life_stage='teenager']").show()
```

```
+-----+---+-----+  
|first_name|age|life_stage|  
+-----+---+-----+  
|      heo| 13|  teenager|  
+-----+---+-----+
```

Spark makes it easy to register tables and query them with pure SQL.

Spark Structured Streaming Example

Spark also has Structured Streaming APIs that allow you to create batch or real-time streaming applications.

Let's see how to use Spark Structured Streaming to read data from Kafka and write it to a Parquet table hourly.

Suppose you have a Kafka stream that's continuously populated with the following data:

```
{"student_name":"someXXperson", "graduation_year":"2023",  
"major":"math"}  
{"student_name":"liXXyao", "graduation_year":"2025",  
"major":"physics"}
```

Here's how to read the Kafka source into a Spark DataFrame:

```
df = (  
    spark.readStream.format("kafka")  
    .option("kafka.bootstrap.servers",  
"host1:port1,host2:port2")  
    .option("subscribe", subscribeTopic)  
    .load()  
)
```

Create a function that cleans the input data.

```

schema = StructType([
    StructField("student_name", StringType()),
    StructField("graduation_year", StringType()),
    StructField("major", StringType()),
])

def with_normalized_names(df, schema):
    parsed_df = (
        df.withColumn("json_data",
            from_json(col("value").cast("string"), schema))
        .withColumn("student_name",
            col("json_data.student_name"))
        .withColumn("graduation_year",
            col("json_data.graduation_year"))
        .withColumn("major", col("json_data.major"))
        .drop(col("json_data"))
        .drop(col("value"))
    )
    split_col = split(parsed_df["student_name"], "xx")
    return (
        parsed_df.withColumn("first_name",
            split_col.getItem(0))
        .withColumn("last_name", split_col.getItem(1))
        .drop("student_name")
    )

```

Now, create a function that will read all of the new data in Kafka whenever it's run.

```

def perform_available_now_update():
    checkpointPath = "data/tmp_students_checkpoint/"
    path = "data/tmp_students"
    return df.transform(lambda df:
        with_normalized_names(df)).writeStream.trigger(
        availableNow=True
    ).format("parquet").option("checkpointLocation",
        checkpointPath).start(path)

```

Invoke the `perform_available_now_update()` function and see the contents of the Parquet table.

You can set up a cron job to run the `perform_available_now_update()` function every hour so your Parquet table is regularly updated.

Spark RDD Example

The Spark RDD APIs are suitable for unstructured data.

The Spark DataFrame API is easier and more performant for structured data.

Suppose you have a text file called `some_text.txt` with the following three lines of data:

```
these are words
these are more words
words in english
```

You would like to compute the count of each word in the text file. Here is how to perform this computation with Spark RDDs:

```
text_file = spark.sparkContext.textFile("some_words.txt")

counts = (
    text_file.flatMap(lambda line: line.split(" "))
    .map(lambda word: (word, 1))
    .reduceByKey(lambda a, b: a + b)
)
```

Let's take a look at the result:

```
counts.collect()

[('these', 2),
 ('are', 2),
 ('more', 1),
 ('in', 1),
 ('words', 3),
 ('english', 1)]
```

Spark allows for efficient execution of the query because it parallelizes this computation. Many other query engines aren't capable of parallelizing computations.

Conclusion

These examples have shown how Spark provides nice user APIs for computations on small datasets. Spark can scale these same code examples to large datasets on distributed clusters. It's fantastic how Spark can handle both large and small datasets.

Spark also has an expansive API compared with other query engines. Spark allows you to perform DataFrame operations with programmatic APIs, write SQL, perform streaming analyses, and do machine learning. Spark saves you from learning multiple frameworks and patching together various libraries to perform an analysis.

Additional examples

Many additional examples are distributed with Spark:

- Basic Spark: Scala examples
(<https://github.com/apache/spark/tree/master/examples/src/main/scala/org/apache/spark/examples>), Java examples
(<https://github.com/apache/spark/tree/master/examples/src/main/java/org/apache/spark/examples>), Python examples
(<https://github.com/apache/spark/tree/master/examples/src/main/python>)
- Spark Streaming: Scala examples
(<https://github.com/apache/spark/tree/master/examples/src/main/scala/org/apache/spark/examples/streaming>), Java examples
(<https://github.com/apache/spark/tree/master/examples/src/main/java/org/apache/spark/examples/streaming>)

Latest News

Preview release of Spark 4.0 (</news/spark-4.0.0-preview2.html>)

(Sep 26, 2024)

Spark 3.5.3 released (</news/spark-3-5-3-released.html>) (Sep 24, 2024)

Spark 3.5.2 released (</news/spark-3-5-2-released.html>) (Aug 10, 2024)

Preview release of Spark 4.0 (</news/spark-4.0.0-preview1.html>) (Jun 03, 2024)

[Archive \(/news/index.html\)](/news/index.html)



(<https://www.apache.org/events/current-event.html>)

DOWNLOAD SPARK ([/DOWNLOADS.HTML](/downloads.html))

Built-in Libraries:

SQL and DataFrames (</sql/>)

Spark Streaming (</streaming/>)

MLlib (machine learning) (</mllib/>)

GraphX (graph) (</graphx/>)

Third-Party Projects (</third-party-projects.html>)

Apache Spark, Spark, Apache, the Apache feather logo, and the Apache Spark project logo are either registered trademarks or trademarks of The Apache Software Foundation in the United States and other countries. See guidance on use of Apache Spark trademarks (</trademarks.html>). All other marks mentioned may be trademarks or registered trademarks of their respective owners.

Copyright © 2018 The Apache Software Foundation, Licensed under the Apache License, Version 2.0 (<https://www.apache.org/licenses/>).